

Programming with R

What is R ?

R Language is specially designed for Statistical computing and Graph representations

It's IDE makes R more efficient having a centralized platform for Coding, Testing, Debugging and Project management.

MODULE - 1

Key Features

User-Friendly Interface: Includes a script editor, console, environment pane, and plot viewer for efficient coding and analysis.

Data Analysis Tools: Supports data visualization with ggplot2 and interactive apps via Shiny.

RMarkdown Integration: Combine R code and text for reproducible reports in formats like HTML, PDF, and Word.

Debugging Tools: Step-through debugging and error tracking for efficient problem-solving.

Package Development: Tools for creating, testing, and documenting R packages with Git integration.

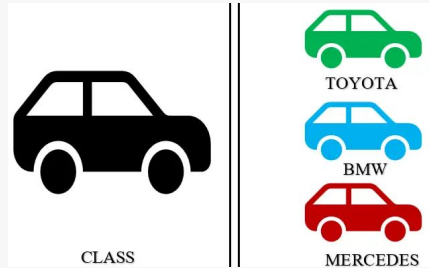
Cross-Platform: Available for Windows, macOS, and Linux.

Extensibility: Integrates with Python, SQL, and supports plugins for additional functionality.

Cloud & Server Options: Offers RStudio Cloud (web-based) and RStudio Server for remote access.

Classes and Objects

Class: A blueprint or template for creating objects. It defines properties (attributes) and behaviors (methods) common to all objects of that type.



Object: An instance of a class. It is a specific realization of the class, containing actual values for the properties defined by the class.

Object Eg:

- ❖ Numeric - Stores Integers, Float etc.....
- ❖ Character - Stores Text and Characters.
- ❖ Logical - Stores Boolean values.
- ❖ Factor - Used to represent Categorical values.
- ❖ Data Frames - Objects that represent tabular data in rows and columns.

Categorical data represents categories or groups of data

- ★ Nominal - Categories with no inherent order or ranking.
Eg: Color, Gender, Country names, etc.....
- ★ Ordinal - Categories with a specific order or ranking, intervals between them are not necessarily uniform.
Eg: Educational levels, Customer satisfaction ratings, etc.....

Object Creation - Objects are created while a value is assigned to the variable using the assignment operator.

' <- ' or ' = '.

- ★ `numeric_object <- 10.5`
- ★ `character_object <- "Data Science"`
- ★ `logical_object <- TRUE`

Operators

Operators are symbols or keywords used to perform operations on variables and values. They can be categorized into different types based on their functionality.

1. Arithmetic operators - used to perform arithmetic operations like addition, subtraction, multiplication, division, modulo division, exponent/power. (+, -, *, /, %, ^)
 - `result_add <- 5 + 3`
 - `result_sub <- 7 - 2`
 - `result_mul <- 6 * 8`
 - `result_div <- 15 / 3`
 - `result_mod <- 15 % 3`
 - `result_pow <- 7 ^ 5`

2. Relational operators - used to compare values and return logical results. (==, !=, <, <=, >, >=)

- `result_lesser_than <- 4 < 6`
- `result_equals_to <- 5 == 5`
- `result_greater_than_equals_to <- 3 >= 9`

3. Logical operators - used to perform operations on logical values (boolean) like AND, OR, NOT. (&, |, !)

`result_and <- TRUE & FALSE`

`result_or <- TRUE | FALSE`

`result_not <- ! TRUE`

4. Assignment operators - used to assign values to the variables, it is used to store and retrieve data from the variable. ('<-' , '=' are the symbols used)

```
variable <- value(s)
```

```
stats <- c(mean, median, mode)
```

5. Miscellaneous operator - used to check if an element is present in the vector or not. ('%in%' is operator used)

```
vector_check <- 7 %in% c(1,3,5,7,9)
```

returns TRUE if the value is present in the vector, FALSE otherwise.

Vectors

A vector is simply a list of items that are of the same type.

To combine the list of items to a vector, use the `c()` function and separate the items by a comma.

```
Eg: data_science <- c("Mathematics, Statistics, Computer science")  
    semesters <- c(2,4,6,5,3,1)
```

- To find the length of a vector use the `length()` function.
- To sort the vectors use the `sort()` function.

Indexing in R starts with 1 not with 0.

To access a value in a vector, matrix, array, list or any other class: `var_name[index_no]`

- To access a value in a vector: `var_name[index_no1, index_no2,.....]`
- There is a concept of negative indexing in R which goes like -1, -2, -3,, You can also use negative index numbers to access all items except the ones specified.
- To change a value in a vector: `var_name[index_no] <- "new_value"`.
- To repeat values in a vector use the `rep()` function, two methods under it are 'each' and 'times'.
- To generate a sequence of number give it in a ratio form and the other method is using a `seq()` function: The `seq()` function has three parameters: from is where the sequence starts, to is where the sequence stops, and by is the interval of the sequence.

Matrices

A Matrix is a two dimensional data structure in R, consisting of rows and columns.

- A matrix can be created with the 'matrix()'function.
- Specify the nrow and ncol parameters to get the amount of rows and columns:
Eg: `matrix1 <- matrix(c(1,2,3,4,5,6), nrow = 3, ncol = 2)`
- You can also create a matrix with strings: Eg: `Matrix2 <- matrix(c("who" , "am" , "i"))`
- You can access the items by using [] brackets. The first number "1" in the bracket specifies the row-position, while the second number "2" specifies the column-position: `matrix1[2,1]`
- The whole row can be accessed if you specify a comma after the number in the bracket: `matrix1[2,]` and vice versa for accessing the whole column.

- To replace individual elements of a matrix: specify the row and column indexes then assign a values to be replaced.
- To replace an entire row of a matrix: specify the row index and leave the column index empty and assign values using the `c()` and vise versa for the replacing entire column elements.
- To remove the existing rows or columns, rows and columns: specify the row or column index using the combine function with a minus as a prefix. `-c(2)`
- To find the number of rows and columns use the `dim()` function.
- To find the length of the matrix use the `length()` function

rbind() – Row Binding: `rbind(object1, object2, ...)`

- `rbind()` (row bind) is used to merge objects by adding rows, provided they have the same number of columns. It's commonly used to stack data vertically.

cbind() – Column Binding: `cbind(object1, object2, ...)`

- `cbind()` (column bind) merges objects by adding columns, ensuring they have the same number of rows.

Function	Combines By	Requirements
<code>rbind()</code>	Rows	Same number of columns
<code>cbind()</code>	Columns	Same number of rows

Arrays

Unlike matrices, arrays can have multiple dimensions (more than two).

- The `array()` function is used to create arrays, with the `dim` parameter specifying their dimensions.

Eg:

```
array1 <- c(1:24)
```

```
array2 <- array(array1, dim = c(4, 3, 2))
```

How does `dim=c(4,3,2)` work?

- The first and second number in the bracket specifies the amount of rows and columns.
- The last number in the bracket specifies how many dimensions we want.

- To access array elements: specify the row position, column position, matrix level i.e; `[3,1,1]`
- To access the whole row or column from an array: `[c(1),,1]` for row and `[,c(1),1]` for column
A comma (,) after c() means accessing the row.
A comma (,) before c() means accessing the column.
- To find out if a specified item is present in an array, use the `%in%` operator:
`2 %in% array`
- To find the number of rows and columns an array, use the `dim()` function.
- To find the length of an array, use the `length()` function.

MODULE - 2

**Non - numeric values,
Decision making and
Loops**

Non-Numeric Values in R

Logical Values:

Represents TRUE or FALSE (Boolean values), commonly used in conditions or comparisons.

Character Values:

Used to store text or string data. Character values are enclosed in quotes (" " or ' ').

Factors:

Used to represent categorical data with fixed levels. Factors are useful for statistical modeling and data analysis.

Lists

A list is a collection of data which is ordered and Mutable.

- To create a list, use the `list()` function.
Eg: `mylist <- list("r", "programming", "for", "analytics")`
- To access the list elements of the list, use the list name with its index position.
Eg: `mylist[2]`
- To change the existing elements from the list, use the list name with the index position and assign value to the variable.
Eg: `mylist[2] <- "program"`

- To find out how many items a list has, use the `length()` function.
Eg: `length(mylist)`
- To find out if a specified item is present in a list, use the `%in%` operator.
Eg: `"Analytics" %in% mylist`
- To add an item to the end of the list, use the `append()` function.
Eg: `append(mylist, "and Inference")`
- You can also remove list items by specifying the index number with a minus as a prefix.
Eg: `mylist[-2]`

- You can access a range of elements by specifying where to start and where to end the range, by using the `:` operator
Eg: `(mylist)[2:5]`
- to join, or concatenate, two or more lists, the most common way is to use the `c()` function, which combines two elements together.
Eg: `list1 <- list("a", "b", "c")`
`list2 <- list(1,2,3)`
`list3 <- c(list1,list2)`

Data Frames

Data Frames are data displayed in a format as a table.

- Data Frames can have different types of data inside it. While the first column can be character, the second and third can be numeric or logical. However, each column should have the same type of data.
- Use the `data.frame()` function to create a DataFrame.
- To get the summary statistics of the data, use `summary(variable_name)`
- We can use single brackets `[]`, double brackets `[[ ]]` or `$` to access elements from a DataFrame.

- To add new rows into the existing Data Frame, use the `rbind()` function.
- To add new columns into the existing Data Frame, use the `cbind()` function.
- To remove rows and columns in a Data Frame, use the `c()` function.
- To find the number of rows and columns in a Data Frame, use the `dim()` function.
- `ncol()` to find the number of columns and `nrow()` to find the number of rows.
- To find the length of the Data Frame, use the `length()` function.
- To combine two or more Data Frame vertically, use the `rbind()` function and `cbind()` function to combine horizontally.

Special Values

Special values such as `Infinity`, `NaN`, `NA`, and `NULL` represent non-standard or missing data. They can significantly affect computations, leading to errors or unexpected results if not handled properly.

1. Infinity (Inf and -Inf): Represents positive or negative infinity, often resulting from division by zero or operations with very large numbers.

Impact on Computations:

- Allowed in numerical operations, but it can distort results.
- Comparisons involving Inf will always evaluate as true or false depending on the context.
- Use the `is.finite()` function to identify finite numbers.

2. NaN (Not a Number): Indicates an undefined or unrepresentable value, often resulting from operations like $0/0$ or $\log(-1)$.

Impact on Computations:

- NaN can propagate through calculations, affecting statistical summaries.
- It cannot be compared to other values directly.
- Use `is.nan()` to detect, then remove NaN values.

3. NA (Not Available / Missing Value): Represents missing or undefined data in vectors or data frames.

Impact on Computations:

- Most operations involving NA return NA, potentially leading to incomplete results.
- Can affect statistical summaries like mean or median.
- Use `na.rm = TRUE` to ignore NA values in calculations.
- Use `is.na()` to detect and then replace NA values.

NULL: Represents the absence of a value or object. Unlike NA, it signifies that the object doesn't exist at all.

Impact on Computations:

- NULL is excluded from calculations and doesn't occupy space in vectors or data frames.
- Functions return NULL if no value is found.
- Use `is.null()` to check for NULL.
- Replace NULL with meaningful values before analysis.

Decision Making Statements

Decision-making statements help control the flow of execution based on conditions.

if Statement:

- Executes a block of code only if a specified condition is TRUE.
- Used for simple one-condition checks.
- Eg: `if (x > 5) { print("x is greater than 5") }`

if-else Statement:

- Executes one block of code if the condition is TRUE, otherwise executes another block.
- Used when two possible outcomes exist.
- Eg:

```
if (x > 5) {  
    print("Greater")  
} else {  
    print("Smaller")  
}
```

if- else if- else Statement:

- Evaluates multiple conditions in sequence and executes the first TRUE block.
- Used when checking more than two conditions.
- Eg:

```
if (x > 10) {  
    print("Big")  
} else if (x == 10) {  
    print("Equal")  
} else {  
    print("Small")  
}
```

switch Statement:

Compares a variable against multiple values and executes the matching case.

Works like a simplified alternative to multiple if-else statements.

Eg:

```
switch(x, "A" = "Excellent", "B" = "Good", "C" = "Average")
```

Nested if Statement:

- An if statement inside another if, used for hierarchical conditions.
- Useful when multiple conditions must be met before executing a block.
- Eg:

```
if (x > 10) {  
    if (x > 20) {  
        print("Very big")  
    }  
}
```

ifelse() Function:

- A vectorized if-else that applies conditions to entire vectors.
- Efficient for conditional element-wise operations in vectors.
- Eg: `ifelse(x > 10, "High", "Low")`

Loop Statements

1. for Loop:

- The for loop iterates over a sequence (vector, list, or range).
- It is used when the number of iterations is known in advance.
- The loop variable takes the value of each element in the sequence one by one.
- It reduces redundancy when performing repetitive operations.

2. while Loop:

- The while loop runs as long as a specified condition is TRUE.
- It is used when the number of iterations is not known beforehand.
- The loop body executes repeatedly until the condition becomes FALSE.
- A counter is often used to avoid infinite loops.

3. repeat Loop:

- The repeat loop runs an infinite number of times until explicitly stopped.
- It does not check any condition at the start, unlike for and while.
- A break statement must be used inside the loop to terminate execution.
- It is useful when we do not know when the condition will be FALSE.

Scoping

Scoping in R determines where and how variables are looked up. R follows lexical scoping, meaning a variable is first searched in the function's local environment and then in the parent (global) environment.

```
x <- 10
local_func <- function() {
  x <- 5
  return(x)
}
print(local_func ())
print(x)
```

Local Scope: Variables declared inside a function are local to it; Global Scope: Variables declared outside functions can be accessed anywhere; Parent Scope: If a variable is not found locally, R looks in the parent environment.

Argument Matching

Argument matching allows functions to handle parameters in different ways, including positional matching, named matching, and partial matching.

```
my_function <- function(a, b, c) {  
  return(a + b + c)  
}  
  
print(my_function(2, 3, 4))  
  
print(my_function(b = 3, c = 4, a = 2))  
  
print(my_function(a = 2, b = 3, c = 4))
```

MODULE - 3

Numeric Variables and Categorical Variables

Analyzing Customer Purchases in a Supermarket:

A supermarket chain is analyzing customer purchases to improve sales strategies. They collect data from 500 customers, including their age, total spending, payment method, and membership status. The goal is to identify purchasing trends based on different customer attributes.

Numerical Variables (Quantitative):

- Age of Customer (in years): A continuous numerical variable that helps identify shopping trends among different age groups.
- Total Spending (in dollars): Another continuous numerical variable showing how much each customer spent during their visit.
- Number of Items Purchased: A discrete numerical variable indicating how many products a customer bought.

Categorical Variables (Qualitative):

- Payment Method (Cash, Card, Digital Wallets, etc.): A categorical variable representing different payment types.
- Membership Status (Member / Non-Member): A categorical variable showing whether a customer is part of the supermarket's loyalty program.
- Product Category Purchased (Groceries, Electronics, Clothing, etc.): A categorical variable classifying customer purchases into predefined groups.

Analysis & Interpretation:

Identifying Trends Using Numerical Variables:

- Older customers (above 40) tend to spend more, as seen with Customer C002 and C004.
- Customers buying a higher number of items generally have higher total spending, showing a positive correlation.

Identifying Trends Using Categorical Variables:

- Customers using digital payments (cards, wallets) often spend more than those using cash.
- Members of the supermarket loyalty program tend to spend more and buy more items.

Real-World Application:

- The supermarket can offer discounts to non-members to encourage sign-ups.
- Targeted promotions can be created for different age groups or payment methods based on purchasing behavior.

Univariate and Multivariate Data

Univariate Data: Univariate data refers to a dataset that contains only one variable.

Eg: Examining the height of students in a class (Height in cm: 150, 160, 165, 170, 175).

- Compute summary statistics (mean, median, mode, variance).
- Create visualizations like histograms, box plots, and bar charts.
- Perform basic statistical analysis like normality checks.

Multivariate Data: Multivariate data consists of multiple variables observed for each individual or object.

Eg: Analyzing students' height, weight, and age together in a dataset.

- Compute correlations between variables.
- Use scatter plots and pair plots for visualization.
- Perform multivariate statistical techniques like Principal Component Analysis (PCA) and regression analysis.

Feature	Univariate Data	Multivariate Data
Number of Variables	One variable at a time	Two or more variables
Example	Examining only height of students	Examining height, weight, and age together
Visualization	Histogram, Box plot	Scatter plots, Heatmaps, PCA
Purpose	Understanding the distribution	Finding relationships between variables
Application	Basic statistics, trend analysis	Predictive modeling, clustering, regression

Outliers

Identifying Outliers

Most of the 50 exam scores fall between 60 and 90

Two scores are 20 and 110. These two values (20 and 110) are clearly outliers, as they deviate significantly from the general distribution of the dataset.

If Outliers Are Included:

- The average may be misleading
- A single high score (110) may falsely inflate the class average, benefiting students overall
- A low score (20) may deflate the average, harming the perceived performance
- Decisions like scholarship eligibility based on average may become unfair

Impact on Measures of Centrality:

1. Mean: The mean is highly sensitive to outliers.

Including 20 (very low) and 110 (very high) will pull the average slightly lower or higher, depending on their relative impact.

Example:

Without outliers, average might be ~75

With outliers, average might drop or rise, depending on the weight of each

2. Median: The median is robust to outliers, as it simply takes the middle value of sorted data.

As long as the number of outliers is small, the median remains stable.

3. Mode: Mode is not impacted by extreme values unless the outlier itself is frequent (which is unlikely here).

So, it stays the same.

If Outliers Are Included:

- The average may be misleading
- A single high score (110) may falsely inflate the class average, benefiting students overall
- A low score (20) may deflate the average, harming the perceived performance

If Outliers Are Excluded:

- The average would better represent the majority
- Ensures more equitable decisions for most students

Covariance

Covariance measures the direction of the linear relationship between two variables.

- If both variables increase together, covariance is positive.
- If one increases while the other decreases, covariance is negative.
- If the variables are independent, covariance is close to zero.

$$\text{Cov}(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{n - 1}$$

Positive Covariance: Variables move in the same direction.

Negative Covariance: Variables move in opposite directions.

Zero Covariance: No linear relationship.

Correlation

Correlation measures the strength and direction of a linear relationship between two variables — and it's standardized (no units).

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \cdot \sigma_Y}$$

Correlation r	Strength
+1	Perfect positive
0.7 to 0.9	Strong positive
0.4 to 0.6	Moderate positive
0 to 0.3	Weak positive
0	No correlation
-0.3 to -0.6	Moderate negative
-0.7 to -0.9	Strong negative
-1	Perfect negative

MODULE - 4

Introduction to ggplot2

ggplot2 is a powerful and flexible data visualization package in R. It helps you create beautiful and customizable plots using a structured system based on the Grammar of Graphics.

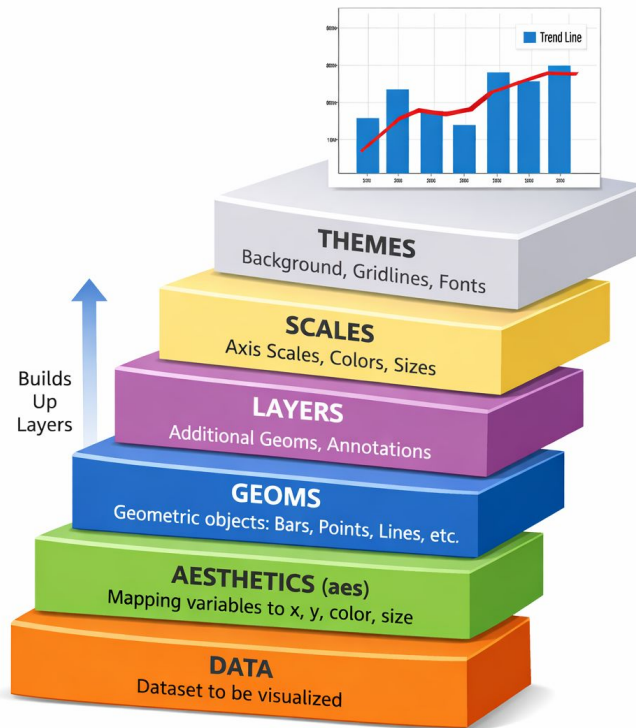
In ggplot2, creating a plot starts with:

- Providing the dataset and mapping variables to aesthetic properties such as the x and y axes.
- Add a geometric object (geom) like points, lines, or bars to define how the data should be visually represented.
- Enhance your plot by customizing themes, labels, colors, and other visual elements to improve clarity and presentation.

```
ggplot(data = <DATA>, aes(x = <X-VAR>, y = <Y-VAR>)) + <GEOM_FUNCTION>()
```

Implementation of Grammar of Graphics in ggplot2

- Data – The dataset to be visualized.
- Aesthetics (aes) – Mappings that link variables in the data to visual properties (like x-position, y-position, color, size).
- Geoms (geom_) – The geometric objects that represent the data visually (e.g., points, bars, lines).
- Layers – Each geom or annotation adds a new layer to the plot, helping build up visual complexity.
- Scales – Control how data values are translated into visual values (e.g., colors, axis scales).
- Themes – Control non-data elements like background, gridlines, and fonts.



Geoms (geom_)

- Geoms define what kind of visual mark you use to represent the data:
- `geom_point()` → scatter plots
- `geom_line()` → line charts
- `geom_bar()` → bar charts
- Each geom requires certain aesthetics (e.g., `geom_point()` needs x and y).

Each call to `geom_`, `stat_`, or `annotate()` adds a layer. Layers can include:

- Geometric layers (`geom_*`)
- Statistical transformations (e.g., `geom_smooth()`)
- Annotations (`annotate()`, `geom_text()`)
- Theming elements (`theme()`)

geom_bar() vs geom_line()

geom_bar(): Used to visualize the count or frequency of categories.

Ideal for showing comparisons between discrete groups.

By default, geom_bar() uses stat = "count" (i.e., it counts how many times each category appears).

Insights Provided:

- Which category is most or least frequent.
- Useful for showing distribution of a categorical variable.
- Great for group comparisons.

geom_line(): Used to visualize trends over time or changes in a continuous variable.

Requires ordered x-values (like dates or time points).

Connects data points with lines to show progression or patterns.

Insights Provided:

- Trends and patterns over time (e.g., increases, peaks, drops).
- Helps identify seasonality, growth, or decline.
- Useful for forecasting and time-based analysis.

Importance of Modifying Themes in ggplot2

Modifying themes in ggplot2 is crucial not just for style, but for enhancing clarity, focus, and communication of data. While ggplot2 provides beautiful default themes, customizing themes allows you to align your visualizations with specific audiences, contexts, and presentation needs.

- **Improves Clarity:** Customizing text size, font, axis labels, and gridlines can make your plot easier to read, especially for presentations or reports. Removing unnecessary elements (e.g., gridlines, background colors) reduces visual noise, making the data stand out.
- **Enhances Aesthetics:** Themed visuals look more professional and consistent, especially when matching brand styles or publication guidelines. Themes like `theme_minimal()` or `theme_classic()` offer clean aesthetics, while `theme_void()` is perfect for maps and custom visuals.
- **Directs Attention:** By adjusting color, layout, and emphasis (bold titles, highlighted annotations), themes help guide viewers' attention to key insights. Eg: enlarging the title or coloring important axis labels makes them pop.
- **Improves Accessibility:** Custom themes can include high-contrast colors and larger fonts, making plots accessible to those with visual impairments.

Annotations in ggplot2

Annotations in ggplot2 are elements you can add to a plot to provide extra context, highlight key data points, or explain patterns/trends. These include text labels, arrows, shaded areas, and custom lines that go beyond the default axes and legends.

1. Clarify Insights: Annotations help you explain what matters most in a plot. Instead of relying on the audience to interpret trends, you can label key events, peaks, outliers, or thresholds.
2. Guide Attention: Well-placed annotations can direct the viewer's focus to specific data points or trends, improving readability and engagement.
3. Tell a Story: By combining annotations with titles and layers, you transform a chart into a data narrative, making it more persuasive and informative.

1. `geom_text()` - Add text labels to points or areas
2. `geom_label()` - Like `geom_text()`, but with a box
3. `annotate()` - Add text, lines, or shapes anywhere
4. `geom_hline()` / `geom_vline()` - Draw reference lines
5. `geom_rect()` - Highlight a region with a shaded box

Significance of heatmaps

Heatmaps are a powerful visualization tool used in exploratory data analysis to represent patterns, correlations, and density within a dataset using color gradients. They are especially helpful when working with high-dimensional or tabular data.

Visualize Patterns Quickly: Heatmaps display values as colors, making it easier to spot trends, clusters, or anomalies at a glance.

Compare Across Categories: You can easily compare values across two categorical variables (like product vs. region) or between features in a matrix.

Reveal Relationships: Heatmaps are often used to show correlation matrices, helping identify strongly correlated variables during feature selection.

Simplify Complex Data: When dealing with a lot of numbers, heatmaps convert complexity into intuitive color patterns.

Layers in ggplot2

The layered grammar of graphics is at the heart of ggplot2. Each component of a plot—data, aesthetics, geometric objects (geoms), statistics, scales, coordinates, and themes—is added as a layer, making it easy to build complex and informative visualizations step by step.

Each `+` in a ggplot call adds a layer. These layers can include:

- Geometric layers: `geom_point()`, `geom_line()`, `geom_bar()`, etc.
- Statistical transformations: `stat_smooth()`, `stat_summary()`
- Labels and themes: `labs()`, `theme_minimal()`
- Annotations: `annotate()`, `geom_text()`
- Scales and legends: `scale_fill_gradient()`, `guides()`

Layers allow you to build complexity gradually.

They separate concerns — data, visual form, and customization are modular.

You can compare multiple datasets or summaries in a single plot.

The core idea in ggplot2 is that a plot is built up from multiple layers, each of which represents a distinct element of the visualization. These layers include:

1. Data & Aesthetics (`aes()`): Define what is being plotted — variables mapped to visual properties like x/y position, color, size, etc.
2. Geometric Objects (`geom_*`): Determine how data is represented visually — points, lines, bars, etc. For instance:
 - `geom_point()` for scatter plots
 - `geom_line()` for trends over time
 - `geom_boxplot()` for distributions
3. Statistical Transformations (`stat_*`): Include statistical summaries — like smoothed lines (`stat_smooth()`), counts (`stat_bin()`), or boxplots.
4. Scales (`scale_*`): Control how data values are translated into visual properties: `scale_x_continuous()`, `scale_color_manual()`, etc. Customize axes, colors, sizes, and more.
5. Facets (`facet_*`): Split data into subsets and plot them in small multiples (subplots) for easy comparison.

6. Themes (theme_*()): Control non-data elements — fonts, background, gridlines, and margins.

7. Coordinates & Labels: Tweak coordinate systems (coord_flip()) and add titles, axis labels, or annotations (labs()).

- Incremental building: Start simple (e.g., geom_point) and gradually add complexity (e.g., geom_smooth, facet_wrap) to test hypotheses and reveal patterns.
- Separation of concerns : Modify a single layer (like the scale or theme) without affecting the rest — makes exploration efficient.
- Multiple geoms/statistics: Overlay different representations (e.g., boxplot + jittered points) to show both summary and raw data.
- Flexibility & Customization: Tweak elements interactively to focus on outliers, clusters, trends, or anomalies.
- Reusability: Use layers programmatically to explore different subsets of the data or visual encodings.